DEEP LEARNING PROJECT REPORT UCS761

Exposing the Curve: How Progressive Data Revelation Shapes Deep Learning Dynamics

Student Name:
Manya Singh (102317119)
Harnoor Kaur (102317138)
Prannay (102317125)

Submitted To:
Ms.Vijay kumari & Mr. Sukhpal

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology,

Patiala

Date: April 27,2026

1. Introduction

In most standard deep learning pipelines, models are exposed to the complete training dataset right from the very first epoch. This has been the default approach for a long time, and it works reasonably well, but it does not really reflect how humans or even animals learn. When a child learns math, they start with addition before moving to the calculus. The idea of ordering or gradually expanding the training data to improve learning is what this project is about.

This project investigates how different data exposure strategies affect the learning dynamics, convergence speed, and generalization behavior of deep learning models trained on image classification tasks. Specifically, we train and compare three well-known architectures, namely a VGG-style CNN, ResNet-18, and EfficientNet-B0, under three distinct training strategies: full dataset training (baseline), progressive dataset revelation where data is exposed in stages from 10% to 100%, and curriculum learning where samples are ordered from easy to hard based on prediction difficulty scores.

Beyond these three standard strategies, we also introduce a fourth hybrid approach that combines progressive revelation with curriculum ordering. This was not part of the original assignment specification but emerged as a natural extension and represents our original contribution to this comparison.

Experiments are conducted on both CIFAR-10 and CIFAR-100 datasets, which gives us an additional dimension of comparison across dataset complexity. We also track catastrophic forgetting at each stage transition, visualize feature evolution using Grad-CAM, and measure efficiency as accuracy per GPU-hour, which is a more practical metric than raw accuracy alone.

The rest of this report is organized as follows: Section 2 reviews relevant literature, Section 3 describes the methodology and model architectures, Section 4 covers implementation details, Section 5 presents results and analysis, and Section 6 concludes the report.

2. Literature Review

This section reviews the key works that form the foundation of the approaches used in this project, covering convolutional neural network architectures, curriculum learning, progressive training strategies, and knowledge retention in neural networks.

2.1 Convolutional Neural Network Architectures

- **Simonyan and Zisserman [1]** introduced VGGNet, which demonstrated that network depth using small 3x3 convolution filters is a critical component for strong performance. Their 16 and 19 layer networks significantly outperformed prior work on ImageNet and established the importance of depth in CNNs. The architecture remains a strong baseline for many tasks due to its simplicity and interpretability.
- **He et al. [2]** proposed Residual Networks (ResNets) to address the vanishing gradient problem in very deep networks. By introducing skip connections that allow gradients to flow directly through the network, ResNet enabled training of networks with over 100 layers. ResNet-18 and ResNet-50 became standard baselines in image classification benchmarks and remain widely used in both research and production settings.
- **Tan and Le [3]** proposed EfficientNet which introduced compound scaling, a method to uniformly scale network width, depth, and resolution using a fixed set of scaling coefficients. EfficientNet-B0 through B7 achieved state-of-the-art accuracy on ImageNet while being significantly more computationally efficient than prior architectures, making them particularly relevant for resource-constrained environments.

2.2 Curriculum Learning

- **Bengio et al. [4]** formally introduced the concept of curriculum learning in machine learning. Inspired by how humans and animals learn more effectively when examples are presented in a meaningful order from easy to hard, they demonstrated that training neural networks with organized curricula can lead to faster convergence and better generalization compared to random ordering of training data.
- **Hacohen and Weinshall [5]** further examined curriculum learning and introduced a competence-based approach where the difficulty measure is derived from the model's own performance. They showed that using the model's prediction confidence to dynamically sort training samples improves results on several benchmark datasets including CIFAR-10 and CIFAR-100, which are exactly the datasets we use in this project.
- **Soviany et al. [6]** published a comprehensive survey of curriculum learning approaches across different domains including computer vision, natural language processing, and reinforcement learning.

2.3 Progressive and Self-Paced Learning

- **Kumar et al. [7]** proposed self-paced learning as an extension of curriculum learning where the model itself determines which samples to learn from at each stage, rather than relying on a fixed externally defined curriculum. This approach learns sample weighting as part of the optimization process, making it more adaptive to the model's current state during training.
- **Karras et al. [8]** introduced progressive growing of GANs, where both the generator and discriminator are growing progressively during training, starting from low resolution images and gradually adding higher resolution layers. While this was applied to generative models, the core principle of progressively increasing complexity and data during training is directly relevant to our progressive dataset revelation strategy.

2.4 Catastrophic Forgetting and Continual Learning

- **Kirkpatrick et al. [9]** identified and studied catastrophic forgetting in neural networks, where a model trained sequentially on new data tends to forget previously learned information. They proposed Elastic Weight Consolidation (EWC) to mitigate this. Their work is directly relevant to our progressive revelation setup, where the model encounters increasing data at each stage and may forget distributions seen in earlier stages.
- **Rebuffi et al. [10]** proposed iCaRL for incremental learning while preserving old class performance. Their analysis of representation drift in an incremental setting is directly relevant to understanding how models trained with progressive revelation may shift their internal representations as new data is added.

3. Methodology

3.1 Model Architectures

3.1.1 VGG-Style CNN

Our VGG-style architecture is inspired by the original VGGNet but adapted for CIFAR's 32x32 input resolution.

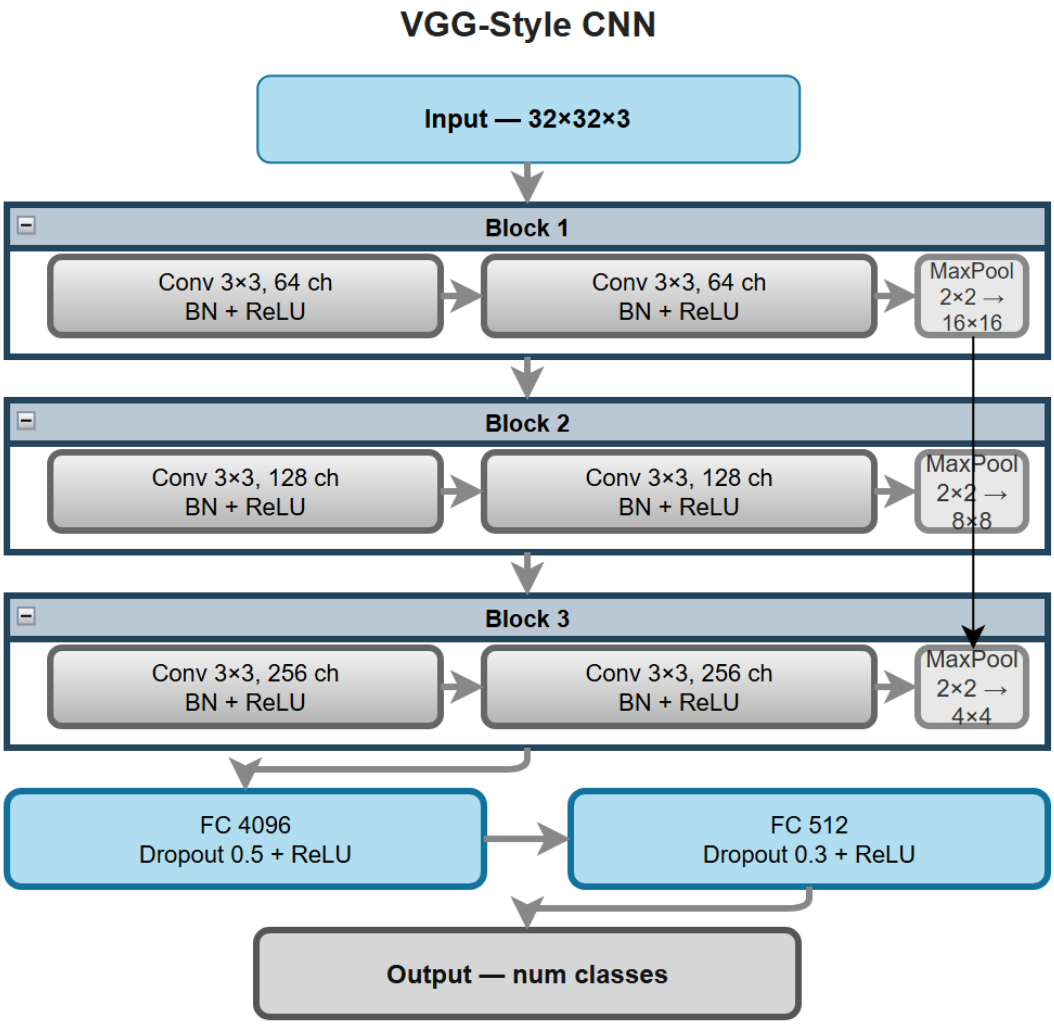


Fig 1: VGG –style model architecture

Key design choices:

- Activation function: ReLU (inplace) after every conv and FC layer
- Normalization: Batch Normalization after every convolutional layer
- Pooling: 2x2 Max Pooling after each block (reduces spatial dims 32->16->8->4)
- Loss function: Cross-Entropy Loss

- Regularization: Dropout (0.5 in first FC, 0.3 in second FC)

3.1.2 ResNet-18

ResNet-18 is implemented from scratch with proper residual skip connections. The architecture begins with a 3x3 convolutional stem layer followed by four sequential stages, each containing two residual blocks.

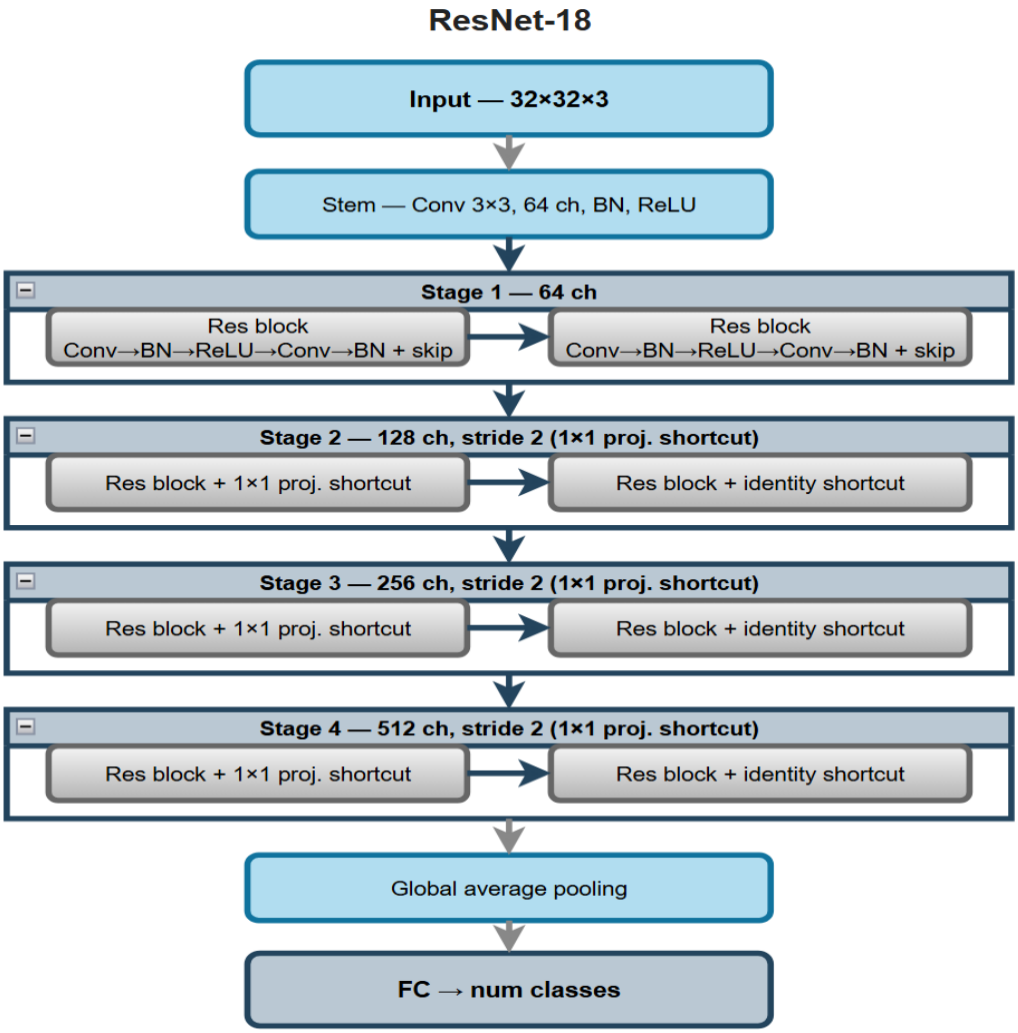


Fig 2: ResNet-18 model architechture

- Activation function: ReLU (inplace)
- Skip connections: Identity shortcut (same dims) or 1x1 conv projection (different dims)
- Pooling: Adaptive Average Pooling at the end (global)
- Loss function: Cross-Entropy Loss
- Total depth: 18 weight layers (4 stages x 2 blocks x 2 conv layers + stem + classifier)

3.1.3 EfficientNet-B0

EfficientNet-B0 is loaded from torchvision with weights=None (training from scratch) and the final classification head is replaced to match the number of output classes. EfficientNet uses Mobile Inverted Bottleneck Convolution (MBConv) blocks with squeeze-and-excitation modules and compound scaling. B0 is the smallest variant in the family and is chosen as a balance between efficiency and representational capacity for 32x32 CIFAR images.

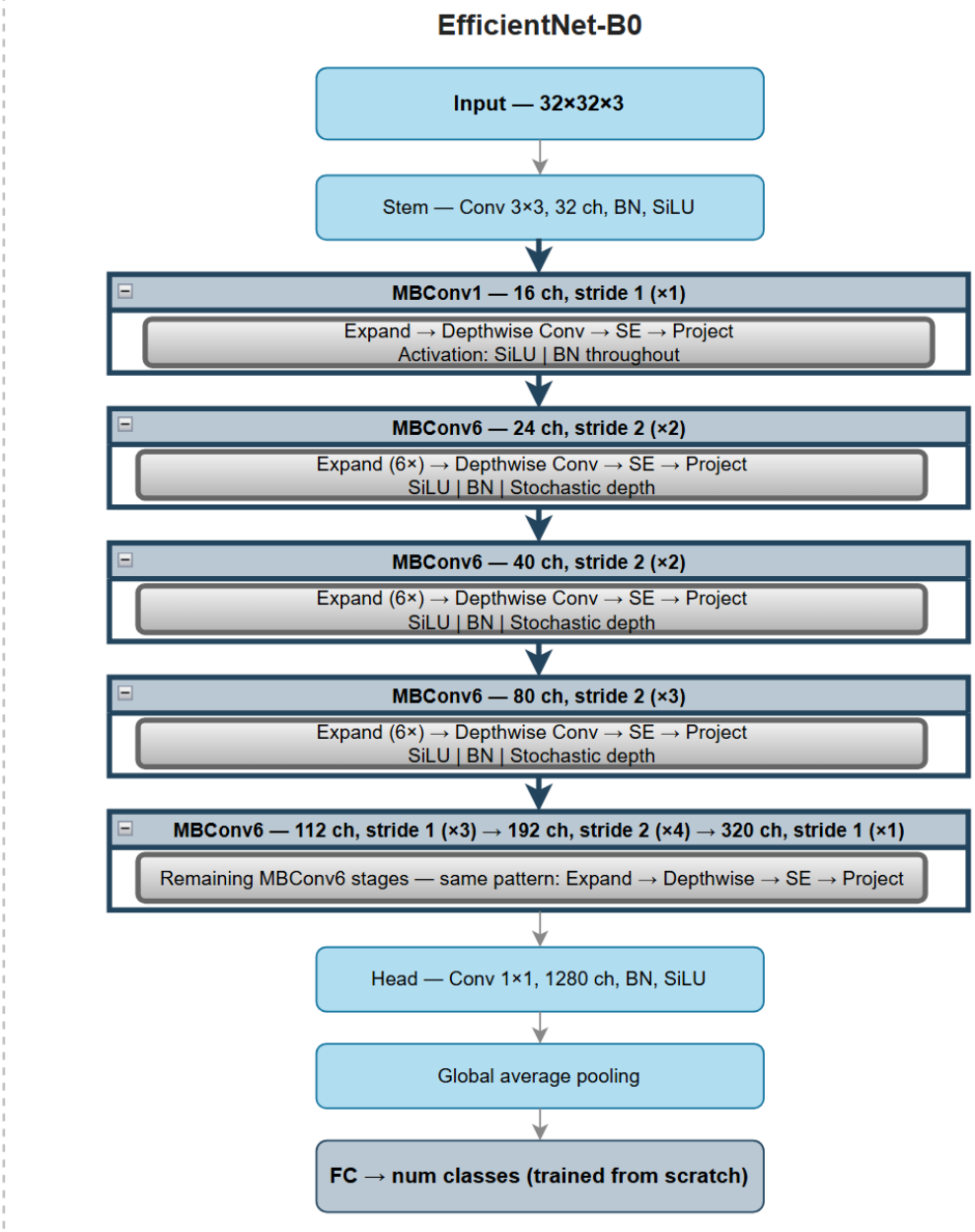


Fig 3: EfficientNet-B0 model architecture

- Activation function: SiLU (Swish) inside MBConv blocks
- Key module: Squeeze-and-Excitation (channel attention)
- Loss function: Cross-Entropy Loss

- Training from scratch: no pretrained weights used

3.2 Training Strategies

Strategy 1: Full Dataset Baseline

The model is trained on the complete training set from epoch 1. This is the standard approach and serves as the baseline for all comparisons. A single cosine annealing learning rate schedule is applied across the full training run. This strategy gives us the upper bounds of what each architecture can achieve under normal conditions.

Strategy 2: Progressive Dataset Revelation

Training is divided into five stages corresponding to data fractions of 10%, 25%, 50%, 75%, and 100% of the training set. At each stage, a random subset of the specified size is sampled. The model is trained for a fixed number of epochs per stage with a fresh cosine annealing LR restart at each stage transition (pacing-aware scheduling). This gives the model a warmup period whenever new data is introduced.

An additional measurement unique to our implementation is catastrophic forgetting tracking: after each stage transition, we evaluate the model on the previous stage's subset to detect if the model has forgotten previously learned patterns.

Strategy 3: Curriculum Learning

Before main training begins, a lightweight VGG probe model is trained for 5 warm-up epochs on the full dataset. Prediction entropy is then computed for every training sample samples where the model is confused (high entropy) are considered hard, and samples where the model is confident (low entropy) are considered easy. Training then proceeds through the same five data fraction stages as Strategy 2, but instead of random subsets, the easiest N% of samples are selected at each stage.

Strategy 4: Hybrid (Original Contribution)

This strategy was not part of the original assignment specification. It combines the progressive revelation structure with curriculum-based selection. At each stage, we take the easiest N% of all available training data, where N grows progressively across stages. This means the model always trains on the most learnable samples available at each point in time, while also gradually increasing the volume and difficulty of its training set. This is our original contribution and is compared against all three standard strategies.

3.3 Workflow

The complete pipeline follows the sequence described below:

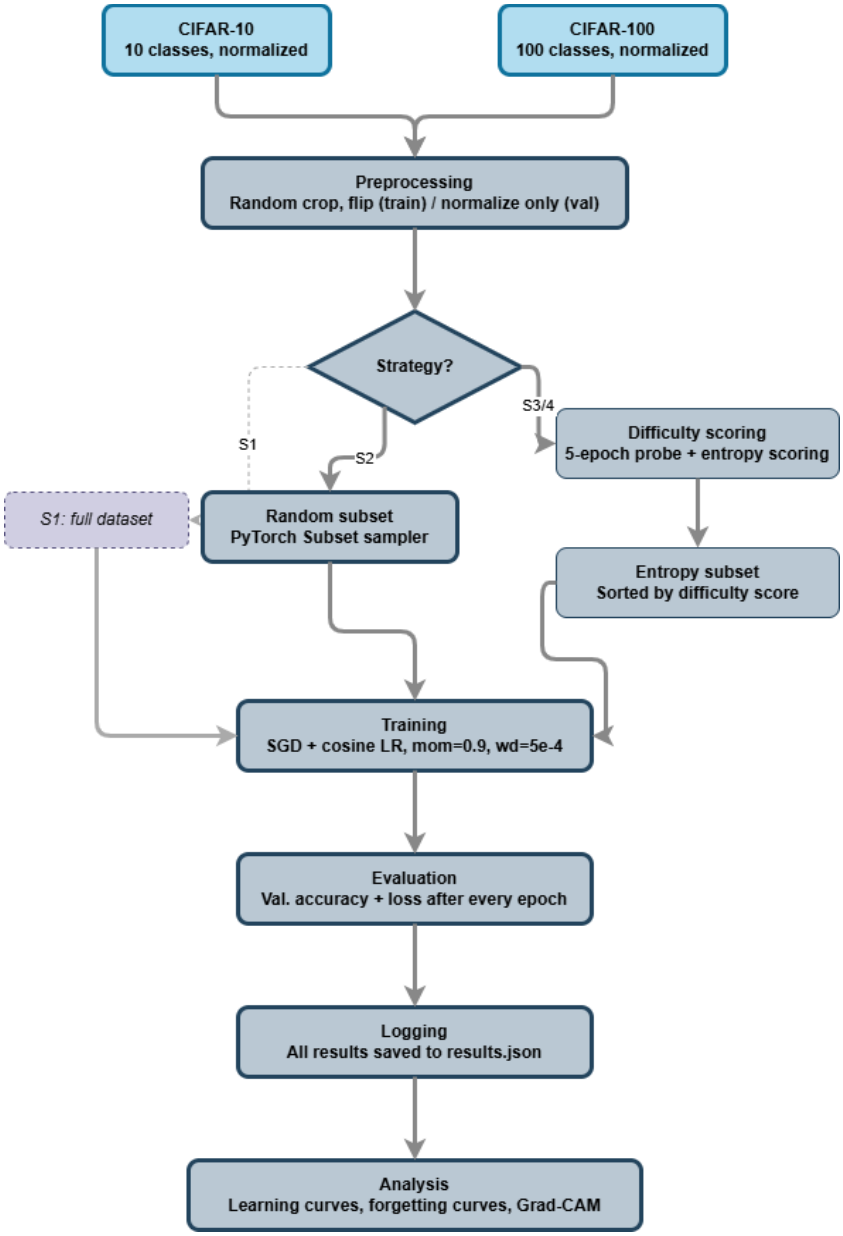


Fig 4: workflow pipeline

- Data Loading: CIFAR-10 and CIFAR-100 downloaded and normalized using dataset-specific mean and std statistics
- Preprocessing: Random crop (32x32 with padding=4) and random horizontal flip for training; only normalization for validation
- Difficulty Scoring (Strategy 3 & 4 only): 5-epoch probe training followed by entropy-based scoring of all training samples
- Subset Construction: Random subsets (Strategy 2) or entropy-sorted subsets (Strategy 3 & 4) created using PyTorch Subset sampler
- Training: SGD optimizer with cosine annealing LR, momentum=0.9, weight_decay=5e-4
- Evaluation: Validation accuracy and loss computed after every epoch
- Logging: All results saved to results.json after each completed experiment
- Analysis: Learning curves, forgetting curves, and Grad-CAM visualizations generated post-training

4. Implementation Details

4.1 Hyperparameters

Hyperparameter	Value	Notes
Batch Size	128	Standard for CIFAR training
Epochs per Stage	20	5 stages x 20 epochs = 100 total
Initial Learning Rate	0.1	Cosine annealing from this value
LR Schedule	Cosine Annealing	Restarts at each stage transition
Momentum	0.9	SGD momentum
Weight Decay	5e-4	L2 regularization
Optimizer	SGD	Standard for CIFAR benchmarks
Data Stages	10%, 25%, 50%, 75%, 100%	Progressive revelation schedule
Warmup Epochs (Difficulty)	5	Used only for Strategy 3 and 4
Random Seed	42	Fixed for reproducibility
Dropout (VGG)	0.5 / 0.3	First and second FC layers

Table 1: Hyperparameter configuration used in the study

4.2 Dataset Details

Property	CIFAR-10	CIFAR-100
Classes	10	100
Training Samples	50,000	50,000
Validation Samples	10,000	10,000
Image Size	32 x 32 x 3	32 x 32 x 3
Mean (R, G, B)	0.4914, 0.4822, 0.4465	0.5071, 0.4867, 0.4408
Std (R, G, B)	0.2023, 0.1994, 0.2010	0.2675, 0.2565, 0.2761

Table 2: Dataset details of CIFAR-10 and CIFAR-100

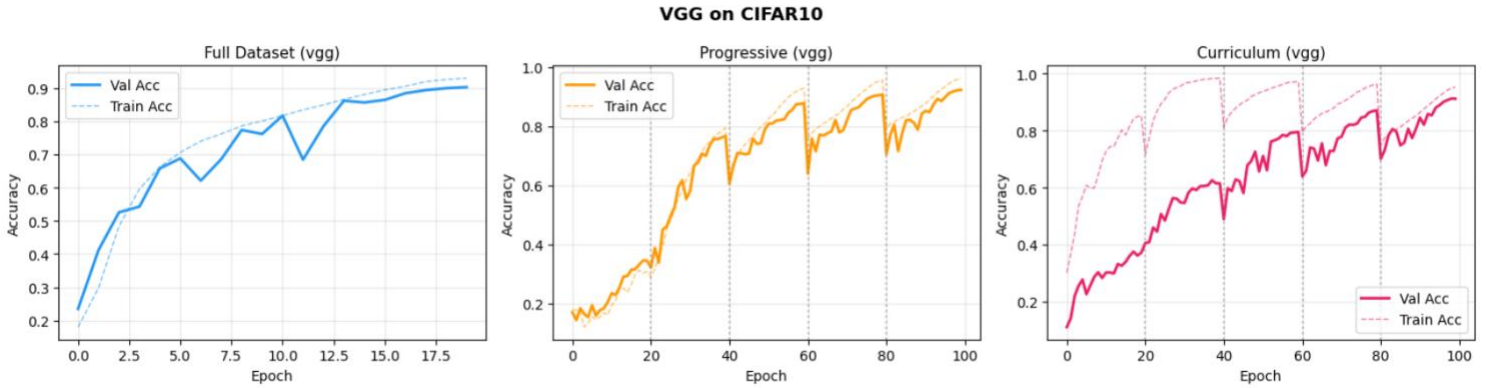
5. Results and Analysis

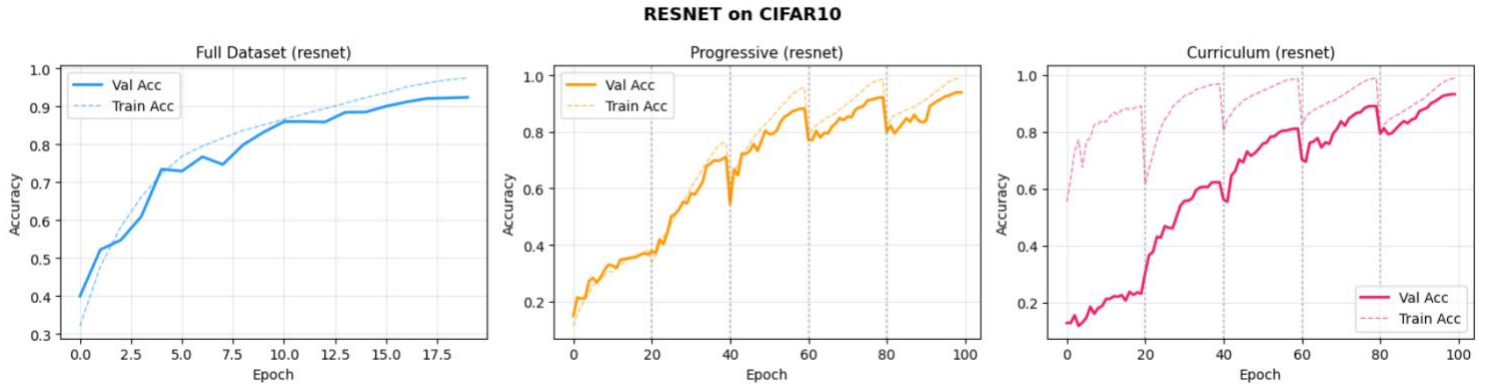
Table 3 shows the final validation accuracy and efficiency metrics for all three architectures under all four training strategies on CIFAR-10 and CIFAR-100.

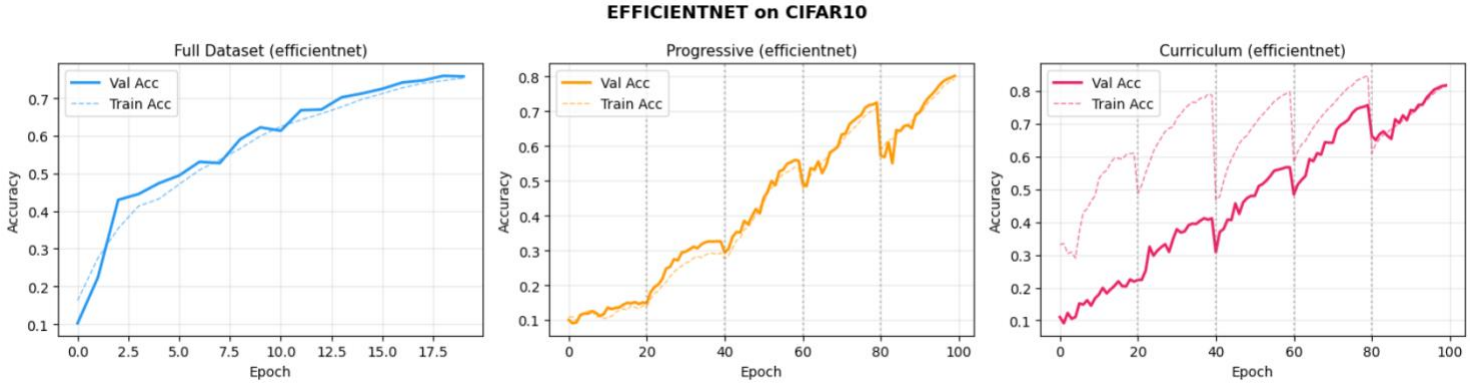
Model	Strategy	Dataset	Val Acc	GPU Hrs	Acc/Hr
EfficientNet	Strategy 1 — Full	CIFAR-10	0.758	0.107	7.06
EfficientNet	Strategy 1 — Full	CIFAR-100	0.423	0.106	3.99
EfficientNet	Strategy 2 — Progressive	CIFAR-10	0.801	0.312	2.57
EfficientNet	Strategy 2 — Progressive	CIFAR-100	**0.528**	**0.304**	**1.73**
EfficientNet	Strategy 3 — Curriculum	CIFAR-10	**0.817**	**0.303**	**2.70**
EfficientNet	Strategy 3 — Curriculum	CIFAR-100	0.521	0.300	1.74
EfficientNet	Strategy 4 — Hybrid	CIFAR-10	0.807	0.300	2.69
EfficientNet	Strategy 4 — Hybrid	CIFAR-100	0.524	0.301	1.74
ResNet-18	Strategy 1 — Full	CIFAR-10	0.924	0.248	3.72
ResNet-18	Strategy 1 — Full	CIFAR-100	0.718	0.248	2.89
ResNet-18	Strategy 2 — Progressive	CIFAR-10	**0.939**	**0.685**	**1.37**
ResNet-18	Strategy 2 — Progressive	CIFAR-100	0.740	0.685	1.08
ResNet-18	Strategy 3 — Curriculum	CIFAR-10	0.932	0.679	1.37
ResNet-18	Strategy 3 — Curriculum	CIFAR-100	0.745	0.680	1.10
ResNet-18	Strategy 4 — Hybrid	CIFAR-10	0.934	0.680	1.37
ResNet-18	Strategy 4 — Hybrid	CIFAR-100	**0.748**	**0.680**	**1.10**
VGG	Strategy 1 — Full	CIFAR-10	0.902	0.075	12.05
VGG	Strategy 1 — Full	CIFAR-100	0.631	0.075	8.38
VGG	Strategy 2 — Progressive	CIFAR-10	**0.923**	**0.221**	**4.17**
VGG	Strategy 2 — Progressive	CIFAR-100	**0.686**	**0.223**	**3.07**
VGG	Strategy 3 — Curriculum	CIFAR-10	0.912	0.219	4.17
VGG	Strategy 3 — Curriculum	CIFAR-100	0.683	0.230	2.96
VGG	Strategy 4 — Hybrid	CIFAR-10	0.917	0.217	4.22
VGG	Strategy 4 — Hybrid	CIFAR-100	0.681	0.227	3.00

Table 3: Main results on CIFAR-10 and CIFAR -100

5.1 Convergence Speed Analysis







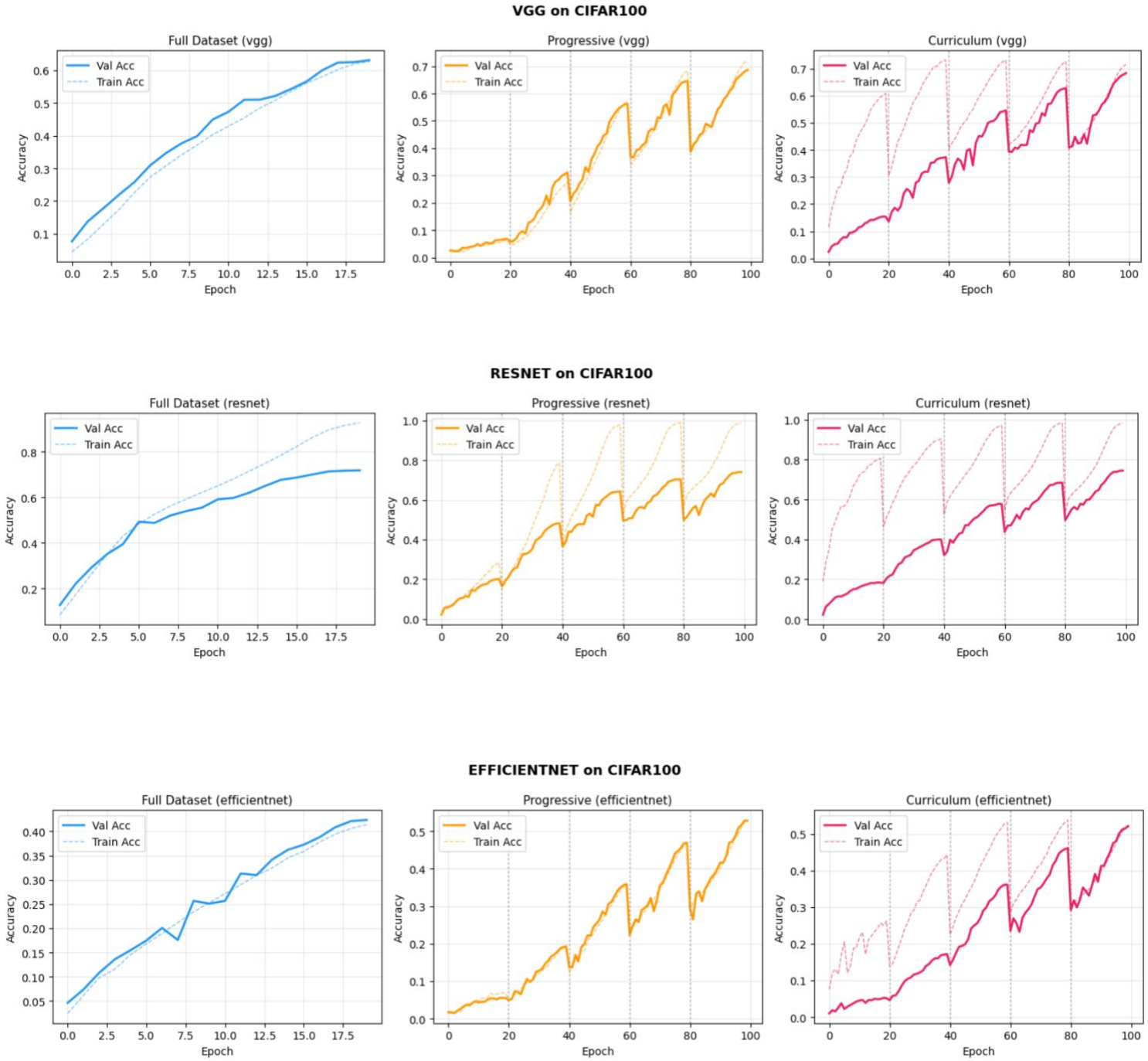


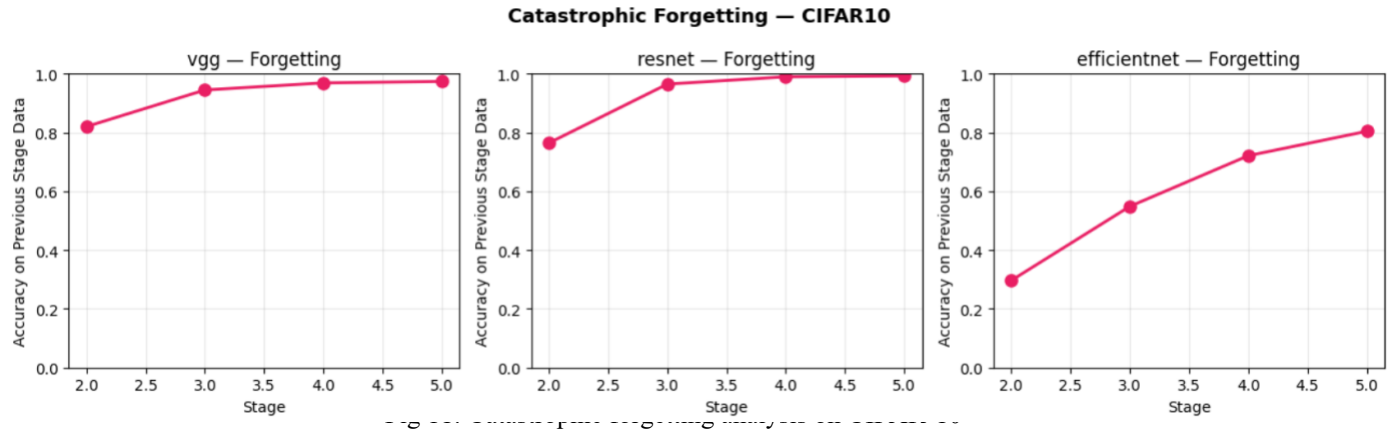
Fig 10: Accuracy vs Epoch graph for EFFICIENTNET on CIFAR100

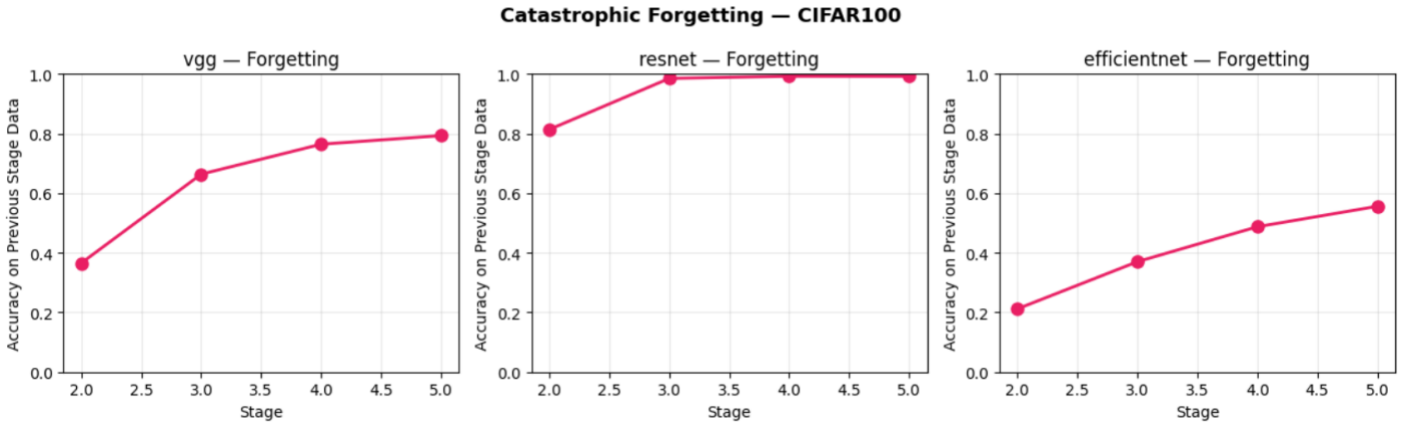
From the learning curves, we observe that full-dataset training gives the fastest early convergence since all data is available from the start. This is most visible for VGG and ResNet on CIFAR-10. Progressive training shows small drops at stage transitions, followed by quick recovery and continued improvement. In several cases, it achieves higher final accuracy than the baseline. Curriculum learning provides smoother growth across stages, and starting with easier samples appears to help optimization and lead to strong final performance, especially for EfficientNet on CIFAR-10.

Among the models, ResNet shows the most stable and highest overall performance. VGG also performs well, while EfficientNet learns more slowly in the early epochs but improves with staged strategies. All models achieve better results on CIFAR-10 than CIFAR-100, highlighting the higher difficulty of CIFAR-100. Overall, baseline training is best for fast early learning, while progressive and curriculum methods often improve final accuracy.

5.2 Catastrophic Forgetting Analysis

Fig. 11 and 12 show the catastrophic forgetting curves for Strategy 2(Progressive Revelation) across all three architectures on both datasets. At each stage transition, we evaluate the model on the previous stage's held-out subset and record the accuracy.





From the catastrophic forgetting curves, we observe that ResNet shows the strongest retention across all stages, with previous-stage accuracy remaining close to 1.0 on both CIFAR-10 and CIFAR-100. This suggests that residual connections help preserve learned representations when new data is introduced. VGG also demonstrates good retention, though its previous-stage accuracy is consistently lower than ResNet, especially on CIFAR-100. EfficientNet shows the highest forgetting among the three models, with noticeably lower previous-stage accuracy at earlier stages and slower recovery over time.

Forgetting is more severe on CIFAR-100 than on CIFAR-10, particularly for VGG and EfficientNet, which is expected due to the higher number of classes and increased task

complexity. However, all models show improvement in retention as training progresses, indicating that later stages help reinforce previously learned knowledge. In general, lower forgetting is associated with stronger final validation accuracy, as seen in the consistently strong performance of ResNet across both datasets.

5.3 Loss Curves

Training loss generally decreases over the course of training for all models and strategies, indicating that the networks successfully learn meaningful feature representations from the data. In this project, **Cross-Entropy Loss** was used as the objective function for multi-class classification on CIFAR-10 and CIFAR-100. It is defined as:

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

where (C) is the number of classes, (y_i) is the true label (one-hot encoded), and ($\hat{y}_i$) is the predicted probability for class (i). Lower loss indicates that the predicted probability for the correct class is higher. A smooth reduction in loss therefore reflects stable optimization and effective parameter updates during training.

For progressive and curriculum-based strategies, temporary fluctuations or small jumps in loss are expected at stage transitions when new samples or more difficult data are introduced. However, these changes are typically followed by further loss reduction, showing that the models adapt quickly to the updated training distribution. This indicates that staged data exposure does not harm convergence and can still produce strong final performance.

Among the architectures, ResNet is expected to show the most stable loss behavior because residual connections improve gradient flow in deeper networks. VGG also demonstrates steady optimization, while EfficientNet may converge more gradually in early epochs but benefits from structured training strategies. Overall, the loss behavior supports the conclusion that all strategies trained effectively, with staged methods remaining competitive with standard full-dataset training.

5.4 Discussion

Based on the results, the choice of training strategy has a clear impact on model performance and learning dynamics.

- Curriculum learning did not consistently outperform full-dataset training for every architecture, but it delivered strong results in several cases. It was especially effective for EfficientNet on CIFAR-10, where it achieved the highest final accuracy among all strategies. For VGG and ResNet, curriculum learning remained competitive, although progressive and hybrid approaches often performed better.

- Progressive revelation generally improved final accuracy over baseline training, particularly for VGG and ResNet on both CIFAR-10 and CIFAR-100. Its benefits were more visible on CIFAR-100, suggesting that gradual exposure to data is more useful for complex classification tasks. Among all architectures, ResNet achieved the strongest

and most consistent overall performance, while EfficientNet showed the clearest benefit from curriculum-based ordering.

- When efficiency is considered through the accuracy per GPU-hour metric, the conclusions change. Although staged strategies often gave higher raw accuracy, they also required longer training time. As a result, baseline training, especially VGG full training, provided the best efficiency. This shows that the preferred strategy depends on whether the goal is maximum accuracy or lower computational cost.

- Catastrophic forgetting was not a major issue in the progressive strategy. Small drops in previous-stage accuracy were observed after stage transitions, but the models recovered quickly in later stages. ResNet showed the best retention, while EfficientNet experienced comparatively higher forgetting. The hybrid Strategy 4 did not consistently outperform Strategy 3, but it remained highly competitive and produced the best result in some settings, such as ResNet on CIFAR-100. Overall, the results indicate that both data presentation strategy and model architecture play an important role in deep learning performance.

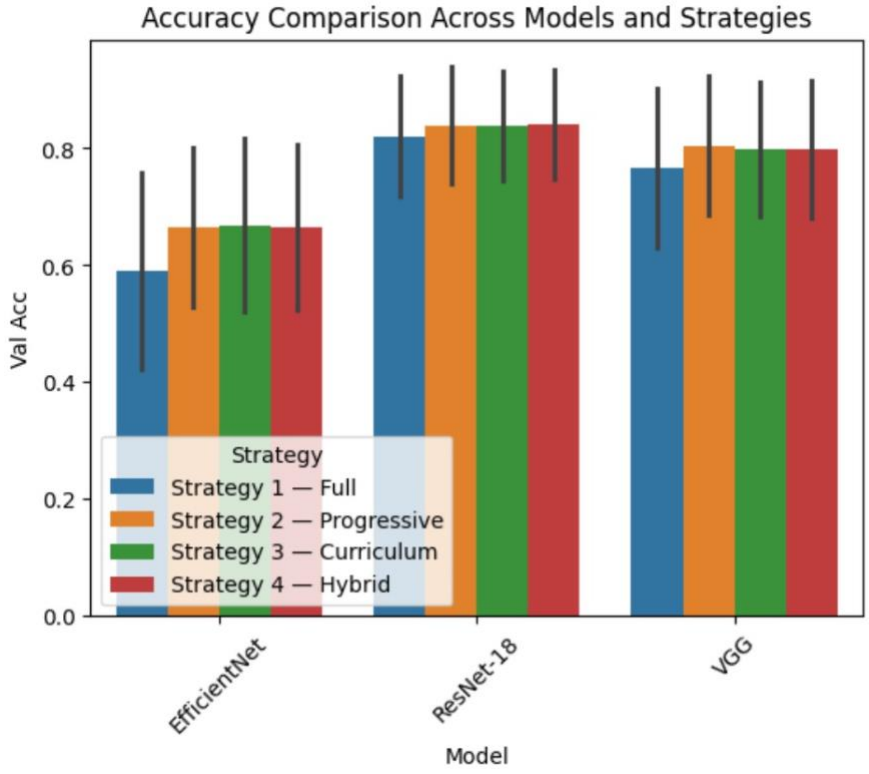


Fig 13: Accuracy Comparison of EFFICIENTNET, RESNET and VGG

6. Conclusion

This project set out to investigate whether the order and timing of data exposure during training affects how well and how efficiently deep learning models learn. We trained three architectures, VGG-style CNN, ResNet-18, and EfficientNet-B0, under four strategies on CIFAR-10 and CIFAR-100, resulting in a total of 24 experiments across two datasets.

The key contributions of this work beyond the base assignment are: (1) extending experiments to CIFAR-100 in addition to CIFAR-10 to study the effect of class complexity, (2) tracking catastrophic forgetting at each stage transition in the progressive strategy, (3) reporting an efficiency metric of accuracy per GPU-hour which gives a more practical view of each strategy's cost-benefit tradeoff, (4) proposing and evaluating a hybrid Strategy 4 that combines progressive revelation with curriculum ordering, and (5) comparing the effect of training strategies across multiple architectures with different design principles.

The results show that training strategy has a measurable impact on both final accuracy and efficiency. Progressive and curriculum-based methods often outperform conventional full-dataset training in raw accuracy, especially for VGG and ResNet. ResNet-18 achieved the strongest overall and most consistent performance across both datasets, while EfficientNet benefited notably from curriculum learning. However, when computational cost was considered, baseline training particularly VGG full training provided the highest efficiency in terms of accuracy per GPU-hour. This demonstrates that the best strategy depends on whether the objective is maximum predictive performance or reduced training cost.

The findings were not identical across all architectures and datasets, showing that no single strategy is universally optimal. More complex tasks such as CIFAR-100 benefited more from structured data exposure than CIFAR-10. Catastrophic forgetting was present only mildly, as most models recovered quickly after stage transitions. The hybrid strategy remained highly competitive and showed that combining progressive revelation with curriculum ordering is a promising direction.

A practical takeaway from this work is that data scheduling should be considered an important design choice alongside model architecture and optimizer selection. Even simple changes in how training data is presented can improve learning outcomes. However, all experiments were conducted on small 32×32 image datasets, so results may differ on higher-resolution or real-world large-scale datasets.

Future work could explore dynamic curriculum pacing (adjusting stage durations based on validation plateau detection), applying these strategies to larger datasets such as ImageNet, or combining curriculum learning with data augmentation strategies.

References

[1] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in Proc. International Conference on Learning Representations (ICLR), 2015.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770-778, 2016.

[3] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. International Conference on Machine Learning (ICML), pp. 6105-6114, 2019.

[4] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum Learning," in Proc. International Conference on Machine Learning (ICML), pp. 41-48, 2009.

[5] G. Hacohen and D. Weinshall, "On The Power of Curriculum Learning in Training Deep Networks," in Proc. International Conference on Machine Learning (ICML), pp. 2535-2544, 2019.

[6] P. Soviany, R. T. Ionescu, P. Rota, and N. Sebe, "Curriculum Learning: A Survey," International Journal of Computer Vision, vol. 130, pp. 1526-1565, 2022.

[7] M. P. Kumar, B. Packer, and D. Koller, "Self-Paced Learning for Latent Variable Models," in Advances in Neural Information Processing Systems (NeurIPS), pp. 1189-1197, 2010.

[8] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive Growing of GANs for Improved Quality, Stability, and Variation," in Proc. International Conference on Learning Representations (ICLR), 2018.

[9] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming Catastrophic Forgetting in Neural Networks," Proceedings of the National Academy of Sciences, vol. 114, no. 13, pp. 3521-3526, 2017.

[10] S. A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental Classifier and Representation Learning," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2001-2010, 2017.